PRESENZ: A FACE RECOGNITION AND ENGAGEMENT DETECTION BASED ATTENDANCE SYSTEM

A PROJECT REPORT ON 6CS1991 - DESIGN PROJECT - II

Submitted by

NITIN HANUMANTHA RAO	(2023BCSE07AED617)
P S LAKSHWIN RUUP	(2023BCSE07AED503)
ROYCE BIJU THOMAS	(2023BCSE07AED354)
GOKUL S	(2023BCSE07AED428)

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (CSE-GENERAL)

Under the Supervision of

Dr. Ganga Holi

Department of Computer Science and Engineering

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BANGALORE - 562106

April - 2026

Department of Computer Science and Engineering

Alliance School of Advanced Computing

Alliance University

Chikkahagade Cross, Chandapura-Anekal Main Road, Bangalore-562106

CERTIFICATE

This is to certify that the Design Project - II work entitled **"Presenz: A Face Recognition And Engagement Detection Based Attendance System"** is the Bonafide work done by **Mr. Nitin Hanumantha Rao (2023BCSE07AED617)**, **Mr. P S Lakshwin Ruup (2023BCSE07AED503)**, **Mr. Royce Biju Thomas (2023BCSE07AED354)**, **Mr. Gokul S (2023BCSE07AED428)** submitted in partial fulfilment of the requirements for the Award of the Degree **Bachelor of Technology in Computer Science and Engineering** during the year 2023-2027.

Dr. Ganga Holi
SUPERVISOR

Dr. Oswalt Manoj S
Head of the Department

External Examiners:

1. Name: _____ Signature: _____

2. Name: _____ Signature: _____

Declaration

This is to declare that the report titled **"Presenz: A Face Recognition And Engagement Detection Based Attendance System"** has been made for the partial fulfilment of the Course Bachelor of Technology in Computer Science and Engineering, under the Supervision of **Dr. Ganga Holi**. We confirm that this report truly represents our work undertaken as a part of our project work. This work is not a replication of work done previously by any other person. We also confirm that the contents of the report and the views contained therein have been discussed and deliberated with the faculty guide.

NAME	REG. NO	SIGNATURE
NITIN HANUMANTHA RAO	2023BCSE07AED617	
P S LAKSHWIN RUUP	2023BCSE07AED503	
ROYCE BIJU THOMAS	2023BCSE07AED354	
GOKUL S	2023BCSE07AED428	

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We are very thankful to our Guide **Dr. Ganga Holi**, Chief Coordinator **Dr. M. Selvam** and Project Coordinator **Dr. Rajasekar R**, Department of Computer Science and Engineering (General) for their sustained inspiring guidance and cooperation throughout the process of this project. Their wise counsel and valuable suggestions are invaluable.

We would like to thank **Dr. Oswalt Manoj S,** Head of the Department, and **Dr. Vullikanti Vivek**, Deputy Director, for their encouragement and cooperation at various levels of the Project.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere and encouragement.

We express our deep sense of gratitude and thanks to the teaching and non-teaching staff at our department who stood with us during the project and helped us to make it a successful venture.

We place highest regards to our parents, our friends and well-wishers who helped a lot in making the report of this project.

NITIN HANUMANTHA RAO
P S LAKSHWIN RUUP
ROYCE BIJU THOMAS
GOKUL S

ABSTRACT

Presenz is an attendance solution for online classrooms that avoids these problems. Presenz incorporates face recognition and monitoring of individual student engagement simultaneously within the same web-based interface. In particular, a face recognizer using the LBPH algorithm will mark attendance automatically using video input from the regular webcam. This is done using a consecutive sequence of four frames as identification. False positives due to momentary misidentification caused by motion blur and partial obstruction have been minimized in the process. On the other hand, a YOLOv8n object detection model will classify each student as being engaged, using the phone, or sleeping. Phone usage will be directly detected using object recognition (COCO class 67). Meanwhile, the act of sleeping is determined through dual triggers: presence of long-term eyelessness through Haar cascade algorithm and the absence of the entire face from a student whose presence has already been established. Each engagement event is tagged to the relevant student through proximity of bounding box detection. Prolonged display of poor behaviour results in change of attendance status from Present to Absent and a timestamped JPEG capture of evidence. There is no need for the use of GPU, external database servers, and specific hardware other than a standard USB camera. The fact is that this solution can be easily applied to current classrooms without making any adjustments to them. After being tested on ten students across five lessons, Presenz managed to get 91.3% attendance and 83.6% engagement detection accuracy with the mAP50 measure. While face recognition applications used for attendance management are capable of tracking whether a student has come or not but nothing more, and behavioural analysis systems generate anonymous data regarding engagement of all participants in the class, Presenz is an open-source application, which combines both functions within one online environment.

Index Terms: Face recognition, attendance management, LBPH, Haar Cascade, YOLOv8, student engagement, behavioural monitoring, Flask, Socket.IO, computer vision.

TABLE OF CONTENTS			
CHAPTER NO.	**TITLE**		**PAGE NO.**
	ABSTRACT		v
1	**INTRODUCTION**		1
	1.1	Background Study	1
	1.2	Statement and Explanation of the Basic Principle	1
	1.3	State of the Art and Work Done	2
	1.4	Objectives of the Project	3
	1.5	Scope of the Project	3
	1.6	Chapter-wise Summary	4
2	**REVIEW OF LITERATURE**		5
	2.1	Literature Review of Selected Papers	5
	2.2	Comparison Table	9
3	**OBSERVATION FROM THE LITERATURE REVIEW**		10
	3.1	Observations from Literature Review	10
	3.2	Contextual Use of Observations in Proposed Work	11
4	**LIMITATIONS OF THE LITERATURE REVIEW**		12
	4.1	Limitations Observed from Literature Review	12
	4.2	Overcoming Limitations in Proposed Work	13

5	**PROBLEM DEFINITION AND PROPOSED RESEARCH METHODOLOGY**		14
	5.1	Problem Statement	14
	5.2	Proposed Research	14
	5.3	Hardware Requirements	15
	5.4	Software Requirements	16
6	**METHODOLOGY**		17
	6.1	Overall System Architecture	17
	6.2	Processing Pipeline	19
7	**MODULE DESCRIPTION**		21
	7.1	Face Recognition Module	21
	7.2	Engagement Detection Module	21
	7.3	Attendance Management Module	22
	7.4	Web Interface Module	23
8	**EXPERIMENTATION**		25
	8.1	Experimental Setup	25
	8.2	Dataset and Enrollment Procedure	25
	8.3	Test Scenarios	26
9	**RESULTS AND DISCUSSION**		29
	9.1	Attendance Accuracy	29
	9.2	Engagement Detection Performance	30
	9.3	Discussion	31
10	**CONCLUSION AND FUTURE WORK**		32
	10.1	Conclusion	32

	10.2	Future Work and Directions	32
11	**REFERENCES**		34
12	**APPENDIX: PLAGIARISM REPORT**		35
13	**APPENDIX: PUBLICATION SUBMISSION PAGE**		36
14	**APPENDIX: SOURCE CODE**		37

LIST OF TABLES

TABLE NO.	NAME OF THE TABLE	PAGE. NO.
1	Comparison Table	9
2	Hardware Requirements	16
3	Software Requirements	17
4	Data Storage Layout	24
5	Technology Stack	25
6	Setup Components	26
7	Attendance Accuracy	30
8	Engagement Detection Accuracy	31
9	Comparison of System performance against the reviewed baselines	32

LIST OF FIGURES

FIG. NO.	NAME OF THE FIGURE	PAGE NO.
1	System Architecture Diagram	19
2	System Flowchart	21
3	Student Registration	27
4	Marking Attendance through Face Detection	28
5	Phone Usage Detection	28
6	Sleeping Detection	29
A1	Plagiarism Report	35
B1	Publication Submission Confirmation	36

LIST OF SYMBOLS, ABBREVIATIONS & NOMENCLATURE

API	Application Programming Interface
COCO	Common Objects in Context
CPU	Central Processing Unit
CSV	Comma Separated Values
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
JPEG	Joint Photographic Experts Group
LBPH	Local Binary Patterns Histograms
LMS	Learning Management System
mAP	Mean Average Precision
ML	Machine Learning
MTCNN	Multi-Task Cascaded Convolutional Network
OpenCV	Open Source Computer Vision Library
RFID	Radio-Frequency Identification
ROI	Region of Interest
SSO	Single Sign-On
UI	User Interface
YOLOv8n	You Only Look Once version 8 nano
UML	Unified Modelling Language

CHAPTER 1

INTRODUCTION

1.1 Background Study

The automation of administrative activities has become inevitable in educational institutions globally. Student attendance is an important activity that helps monitor the participation of students, maintaining their record, and holding them accountable. However, despite its significance, student attendance has remained largely manual in most institutions of learning. The conventional methods of attendance such as roll call using paper, fingerprint recognition systems, and use of RFID cards consume a lot of time and are prone to manipulation through proxy attendance.

A normal lecture may last for about one hour, during which 5 to 10 minutes could be lost because of the roll call process. Apart from wasting time, traditional methods enable students to check themselves out manually without any validation procedure. Fingerprint scanning and RFID card readers rely on physical components, which mean that interaction with the machine will take time.

Face recognition technology offers a method without physical contact and without human assistance in recognizing several people at a time through a simple webcam. Nevertheless, many systems of face recognition for attendance purposes concentrate only on finding and registering the presence of people but not on informing the teacher about what those people are doing next.

This concept forms the basis of Presenz. Academic achievement is dependent on physical presence as well as active participation. If a learner is present physically but not attentive, then they contribute nothing towards their education and class environment. Providing objective information regarding learner participation, rather than presence, to teachers will enable them to take appropriate measures that lead to academic success.

1.2 Statement and Explanation of the Basic Principle

Presenz essentially operates by integrating two major computer vision systems that operate concurrently using only one computer processor. The first system detects facial recognition to record attendance depending on whose present, while the second monitors behavior to determine whether the students are attentive or not. The concept sounds simple enough, but the real challenge lies in its implementation without any additional hardware.

As for the face recognition technique, they rely on a method referred to as Local Binary Patterns Histograms (LBPH). The system first decomposes the face image into several blocks of equal size. For every pixel within these blocks, a code describing the local binary pattern representing the texture is computed. Next, the system constructs the histogram of these local binary pattern codes for every block and concatenates them to form a single feature vector. In order to identify the person, the system uses a chi-squared distance between this vector and the vectors corresponding to the faces seen during the training stage. This yields a distance metric that is converted into the confidence measure.

However, individual frames could sometimes be misleading in scenarios such as when a person's movement is very rapid causing blurring of the image or abrupt change in lighting or a part of his/her face being hidden. For this reason, Presenz will need four consecutive frames to positively match an identity. However, the frames cannot be any frames but rather those that capture the same position of the face in the video stream. This makes perfect sense as this will minimize false positives.

Now, regarding engagement monitoring, we have two parallel pipelines. The first one concentrates on detecting eye presence using the Haar cascade detector for the top part of the face bounding box detected earlier for each student who passed facial recognition. The absence of eyes is interpreted as the student looking downwards either towards their smartphone in case it occurs briefly, or sleeping in case it persists. Another pipeline detects smartphones throughout the entire frame using the smaller variant of YOLOv8 trained on the COCO dataset (YOLOv8n). Whenever there is a phone detected in the vicinity of the student's face bounding box, the detection is considered a case of phone usage and overrides any findings from the eye detection pipeline. This entire setup works purely using the CPU, without the need for specialized hardware. As far as we can tell, this solution will work smoothly even with a complete classroom full of students, though we cannot guarantee its performance under such circumstances. Overall, engagement monitoring seems rather heuristic in nature.

1.3 State of the Art and Work Done

The use of face recognition technology for taking attendance was already studied in research about ten years ago. In my opinion, the earlier studies were based more on techniques such as Eigenface and Fisherface, which are essentially ways to reduce data from a person's face into a lower space for easier comparison. The techniques mentioned were linear projection techniques.
Later on, texture-based techniques such as LBPH emerged as more efficient in terms of lighting and required less computational power.

However, deep learning brought about a paradigm shift. Techniques such as AdaFace (2022) and the lightweight technique proposed by Surantha and Sugijakko (2024) are indicative of the future. While

they provide excellent accuracy on hard benchmarks, even their lightweight versions can leverage GPU support to process large cohorts. In the case of an institution that only has a few students and possesses only CPUs, LBPH is still the way to go.

In parallel, engagement tracking also follows its course. Initially, they attempted face expression analysis to estimate the emotional state. Recently, the focus has shifted to head orientation, gaze direction, or even behavior like cell phone use and sleeping.

The challenge that runs through much of these studies is that they remain at the classroom level. For example, the study will show how many people in the class are attentive at any one time but fails to integrate the data with the individuals themselves. No study combines facial recognition technology with monitoring individuals' behavior and automatic recording of attendance.

1.4 Objectives of the Project

- Automate attendance marking using real-time face recognition.
- Monitor per-student engagement throughout the session to detect phone usage and sleeping behaviour.
- Link detected behavioural events to individual attendance records with timestamped snapshot evidence.
- Operate entirely on commodity CPU hardware without any additional sensors, GPU, or infrastructure.
- Generate downloadable per-subject attendance reports with status, reason, and photographic evidence columns.
- Implement a browser-accessible web interface for instructors to manage students, start sessions, and review reports.

1.5 Scope of the Project

This system is meant to be used in a classroom scenario with a setup of 10-20 students sitting and facing a single webcam. This system has been developed into a locally hosted web application and can be accessed through any browser connected to the same local area network. The current scope of this system includes face recognition attendance marking, attendance demotion through misconduct, eye visibility, phone detection, and CSV report generation.

The software requires no database external to it, and data storage takes place using flat CSV files and folder structure for images. This allows the software to depend little on infrastructure and become

deployable even within organizations lacking cloud access or server capabilities. This software has been tested on 10 subjects in five 5 minute trials each.

1.6 Chapter-wise Summary

- Chapter 1 - Introduction: Presents the background, motivation, basic principle, objectives, and scope of Presenz.
- Chapter 2 - Review of Literature: Reviews ten relevant papers across face recognition, object detection, and student engagement monitoring, followed by a comparison table.
- Chapter 3 - Observations from the Literature Review: Summarises key design-relevant findings from the reviewed literature.
- Chapter 4 - Limitations of the Literature Review: Identifies gaps in prior work that Presenz is designed to address.
- Chapter 5 - Problem Definition and Proposed Research Methodology: States the problem formally and describes the proposed solution, hardware requirements, and software requirements.
- Chapter 6 - Methodology: Describes the overall system architecture, data flow, and processing pipeline.
- Chapter 7 - Module Description: Details each of the four system modules and their interactions.
- Chapter 8 - Experimentation: Describes the experimental setup, dataset, enrollment procedure, and test scenarios.
- Chapter 9 - Results and Discussion: Presents quantitative evaluation results and discusses findings.
- Chapter 10 - Conclusion and Future Work: Summarises contributions and outlines future enhancement pathways.

CHAPTER 2

REVIEW OF LITERATURE

Presenz is based on previous research carried out in three main fields, which include facial recognition technology, object detection in real time, and automatic engagement tracking among students. In this section, we discuss ten studies that have been carried out within the last five years regarding these technologies.

2.1 Literature Review of Selected Papers

Paper 1: AdaFace: Quality Adaptive Margin for Face Recognition

AdaFace, introduced by Kim (2022), is a face recognition system based on a proposed quality-aware margin loss. The intuition behind AdaFace is that depending on the quality of the image provided as input, the strategy used to give more importance to either hard or easy cases in training the model should be adjusted accordingly. The image quality in AdaFace can be determined via the feature norm, which indicates that high-norm embeddings belong to high-quality images and should give emphasis to hard samples, whereas low-norm embeddings represent poor-quality images and should give more importance to easy samples. This method does not involve an independent quality detector; rather, an end-to-end training approach is followed. AdaFace achieves superior performance compared to previous methods based on margins and has a specific edge in dealing with low-quality and low-resolution images, which makes it very suitable for classroom applications using webcams. However, AdaFace requires GPU acceleration when deploying its backbone networks; therefore, alternative methods should be considered, e.g., LBPH with CPU only.

Paper 2: Lightweight Face Recognition-Based Portable Attendance System with Liveness Detection

Surantha and Sugijakko (2024) have developed a lightweight face recognition attendance system with liveness detection to thwart any attempts at face spoofing. The authors utilize MobileNetV2 as the network architecture, which was trained using transfer learning, thereby allowing face recognition to occur in real time using a Raspberry Pi, which has minimal processing capabilities. The process of liveness detection occurs before recognition, ensuring that spoof attacks of Level 2, like face printouts or screen replays, are thwarted since they would not be recognized by a face recognition-only pipeline. The authors test their model using real attendance cases and find that the system is able to achieve high accuracy levels of face recognition while remaining completely on a CPU-level device. The

authors show that accurate and safe face recognition is possible using a portable device, thus providing inspiration for the creation of the Presenz app.

Paper 3: RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild

Deng (2020) introduced RetinaFace, which is a one-stage, multi-level face detector designed to jointly detect boxes, five key-point landmarks, and dense vertices of 3D faces in a single forward computation. With a backbone Feature Pyramid Network, RetinaFace can detect faces under different scales and poses, scoring 91.4% AP on the hard subset of the popular WIDER FACE dataset, thereby outperforming all two-stage detectors before. In addition, its compact version MobileNet works in real-time on CPU input VGA-sized images. Compared with the older three-stage MTCNN architecture, RetinaFace and Proposal Network, Refine Network, and Output Network are superior in accuracy thanks to the multi-task learning mechanism, not depending on the cascade of different networks. Presenz adopts the lighter Haar Cascade face detector rather than RetinaFace to avoid any new dependency on deep learning, even at the cost of slightly decreased detection accuracy. In the case of the controlled indoor environment of the classroom with mostly frontal faces up-close, Haar Cascade proves enough in terms of detection; however, RetinaFace is a potential improvement choice.

Paper 4: Ultralytics YOLOv8

In 2023, Jocher introduced YOLOv8, with the innovation of an anchor-free detection head and an advanced backbone module called C2f, which has a better gradient flow than the backbone module called C3 in YOLOv5. The nano version of YOLOv8, namely YOLOv8n, can score 37.3% mAP50-95 for COCO objects and run efficiently on CPU, enabling it to be used in real-time for embedded and edge devices. Presenz employs YOLOv8n with COCO pre-trained weights to recognize phones in the COCO object class 67 from the classroom frame. Because phones are abundant in the COCO dataset, the pre-trained model is effective in recognizing them in classroom images without additional fine-tuning.

Paper 5: What is YOLOv8: An In-Depth Exploration of Internal Features

Yaseen (2024) provided a detailed architectural review of YOLOv8, looking at the C2f backbone, decoupled head for detecting objects, and distribution-focused losses. According to the review, it was found that the improved feature hierarchy provided by the C2f backbone results in better detection of smaller or partially covered objects than the previous YOLO iterations. It is significant when dealing with the presence of mobile phones in the classroom, where there is a possibility that the phone is not entirely visible to the camera or is partially concealed within the student's lap or their hands. Furthermore, the decoupled head design has been proven to reduce conflict between classification and

regression processes, thereby ensuring accurate class-wise confidence score - critical for Presenz's 0.45 confidence level.

Paper 6: An Intelligent System for Monitoring Students' Engagement in Large Classroom Teaching

The researchers Pabba and Kumar (2022) created an engagement classifier based on facial expressions for larger classes based on video feed data from ordinary cameras. In their work, they utilized a CNN classifier that mapped facial actions to engage scores and generated class-wise engagement estimates without the need to identify any of the students in the process. On being tested on video footage from 30-60 students' classes, they concluded that automated engagement estimates were possible even without using specialized equipment. Nevertheless, the major drawback of their work was that it could not map the engagement states of particular identified students.

Paper 7: Real-Time Attention Monitoring System for Classroom

According to Trabelsi (2023), a deep learning-based approach was developed for the purpose of classroom attention tracking, which was able to detect mobile phone use, dozing off, and distraction through standard laptop hardware. The model used in this research comprised a customized CNN trained with behaviour images in the classroom environment and was validated using a sample of 20 participants in controlled settings, demonstrating 88.4% accuracy in classifying the three engagement types. Thus, the results of the research prove that the behavioural indicators of engagement can be distinguished on video and confirm the efficiency of Presenz' engagement tracking model. However, the developed approach is incapable of recognizing faces.

Paper 8: Recognition of Student Engagement State Using Deep Transfer Learning

Deep Transfer Learning (MobileNetV3 Fine Tuning), as used by Ikram et al. (2023), in engagement classification tasks has shown successful generalisation to unseen students with relatively few training samples. In their research, they have observed that using as little as 500 pictures per engagement category is sufficient to achieve engagement classification success rates greater than 85%, showing the practicality of using smaller data sets. Most importantly, they have observed that variations in visual appearance, especially due to different lighting conditions and inter-session changes like hairstyle or wearing glasses, severely hamper the task performance.

Paper 9: Student-Engagement Detection in Classroom Using Machine Learning

The authors Alruwais and Zakariah (2023) performed an assessment of machine learning algorithms for detecting student engagement based on SVM, Random Forest, KNN, and Gradient Boosting. Based on their findings, it can be concluded that behavioural features, such as body pose and device interaction, work better than facial expressions for student engagement detection. Among the evaluated classifiers, the best accuracy of 91.2% was provided by the Random Forest model. This conclusion makes it possible to draw the following parallelism while designing the algorithm for engagement detection, the authors of Presenz decided not to rely on facial expression recognition due to the need for extensive data collection for this task.

Paper 10: Facial Recognition Attendance System Using Flask and OpenCV

Krishnaprasad (2023) created a facial recognition attendance system based on Flask and OpenCV, employing LBPH to establish that such a combination of technologies is feasible and adequate for actual implementation within a classroom environment on normal computers. The authors reached 87% recognition success on their dataset consisting of 25 students using a regular laptop camera. Nevertheless, the presented system does not implement any engagement tracking features; it lacks a feature of creating photographic proof and offers merely binary attendance recognition (presence vs. absence). This paper directly serves as the baseline extended by Presenz with its additional functionality.

2.2 Comparison Table

Table.1: Comparison Table

S.No	Author / Paper	Year	Research Methodology	Performance Metrics	Advantages	Disadvantages
1	Kim - AdaFace	2022	Quality-adaptive margin loss; image quality proxy	SoTA on IJB-B, IJB-C, IJB-S, TinyFace	Handles degraded/low-quality images; quality-aware training	GPU required for practical throughput; not CPU-deployable at scale
2	Surantha & Sugijakko - Lightweight Attendance with Liveness Detection	2024	MobileNetV2 transfer learning; liveness detection on Raspberry Pi	Real-time CPU inference; spoof attack detection	Lightweight; anti-spoofing; portable CPU deployment	No engagement monitoring; no per-student identity linking
3	Deng, Guo - RetinaFace	2020	Single-shot FPN; multi-task face box landmark localization	91.4% AP on WIDER FACE hard subset	Single-pass; multi-scale; CPU-capable MobileNet variant	Heavier than Haar Cascade; extra dependency for Presenz scope
4	Jocher - YOLOv8	2023	Anchor-free head & C2f backbone	37.3 mAP50-95 on COCO	Fast, CPU-capable nano variant	Pre-trained only; custom training for niche classes
5	Yaseen - YOLOv8 Analysis	2024	Architectural analysis	N/A (analysis paper)	Confirms C2f advantage for small objects	Not an implementation paper
6	Pabba & Kumar	2022	CNN facial expression analysis	Class-level engagement estimation	No specialised hardware required	No individual student identity linking
7	Trabelsi	2023	Custom CNN on classroom video	88.4% engagement accuracy	Real-time, laptop-deployable	No face recognition, no attendance link
8	Ikram et	2023	MobileNetV3 transfer learning	>85% engagement accuracy	Generalises with small dataset	No identity linking, degraded with appearance change
9	Alruwais & Zakariah	2023	ML classifiers on behavioural features	91.2% (Random Forest)	Behavioural cues outperform expression	No identity recognition, offline only
10	Krishnaprasad	2023	Flask, OpenCV & LBPH	87% attendance accuracy	Practical, CPU-only, lightweight	No engagement monitoring, no demotion
11	Presenz (This Work)	2026	LBPH, YOLOv8n & Haar Eye	91.3% att., 83.6% mAP50 eng.	Full pipeline: ID, engagement & demotion	Limited to small cohort, frontal faces only

CHAPTER 3

OBSERVATION FROM THE LITERATURE REVIEW

3.1 Observations from Literature Review

The following observations were drawn from the ten reviewed papers and directly influenced design decisions in Presenz:

- Observation 1 - Tradeoff Between Accuracy and Computing Power: Deep learning-based face recognition techniques that prioritize accuracy, including AdaFace, exhibit high precision when applied to both low- and high-quality image sets; however, they necessitate the use of GPU technology. According to Surantha and Sugijakko (2024), it is feasible to deploy an attendance recognition system that uses MobileNetV2 transfer learning on CPUs. Although LBPH is less accurate than other methods, it can be employed in smaller classrooms where the number of students is constant and obtains satisfactory results (87-91%).
- Observation 2 - Detection Efficiency: The use of Haar cascade-based face detection is efficient and sufficient for classroom settings where the environmental factors are relatively controlled. Although RetinaFace boasts excellent accuracy through its Feature Pyramid Network backbone, this comes at an unnecessary cost for Presenz, considering the favorable setting in which the facial detection occurs.
- Observation 3 - Suitability of YOLOv8n for the task: YOLOv8n is a stable model which runs efficiently even on CPU. YOLOv8n detects mobile phones using pre-trained COCO weights without requiring any customization through training for this purpose. Richer hierarchy of features in the backbone C2f contributes towards detecting mobile phones even when they are partially occluded by objects.
- Observation 4 - Individual Identity Connection Gap: The current systems that monitor engagement have been designed to give results at the level of the class but cannot determine which exact student is the disengaged one. There are no current systems that can connect the instances of engagement to the individual's attendance record.
- Observation 5 - Technology Stack Validation: Both Flask and OpenCV have already been proven to be a viable and lightweight combination of technologies to be used for real-time attendance management through face recognition. Implementing this technology stack in Presenz means that the fundamental technology decisions are based on proven practices.
- Observation 6 - Indicators of Disengagement Over Expression: The study conducted by Alruwais and Zakariah (2023) found that indicators like head posture and device utilization are more

accurate indicators of engagement than facial expressions. This is consistent with the Presenz solution to employ eye visibility heuristic and device identification for detecting engagement, and not just facial expressions.

- Observation 7 - Quality of Enrollment Is Important: According to Ikram (2023), the variability in appearance adversely affects the recognition process. In order to eliminate this problem, capturing 60 images for each enrolled individual as opposed to using the minimum number is recommended.
- Observation 8 - CPU Capability Verification: Several systems examined, Krishnaprasad (2023), Trabelsi (2023), and YOLOv8n nano, separately proved face recognition and object detection to be feasible with a CPU. Presenz uses the capabilities of both in one pipeline, and the throughput test results show that this is possible to do with the CPU as well.

3.2 Contextual Use of Observations in Proposed Work

Decisions on using LBPH instead of deep model classifiers and Haar Cascade instead of RetinaFace were made directly due to Observations 1 and 2. Although AdaFace will have better accuracy and RetinaFace better detection reliability compared to other models, the fact that the software has to run entirely on CPU limits its usability in institutions and makes the accuracy reduction acceptable.

Based on Observation 3, YOLOv8n was chosen as the backbone for phone detection. The pre-trained COCO weights make any manual annotations unnecessary, while the anchor-free architecture makes configuration easy. The 87.2% AP50 of phone detection in Presenz supports this decision.

Observation 4 forms the basis of motivation behind the whole Presenz system. To the best of our knowledge, there is no prior effort that integrates face-based identification with monitoring of individual engagement levels and automated marking for low engagement levels in terms of attendance. Observation 5 facilitated speedy development using an already proven stack. The routing capabilities of Flask and the real-time event emission capabilities of Flask-Socket.IO offer the exact functionality required by the dashboard without adding any extra baggage.

Observations 6 and 7 influenced the exact specifications of the engagement detection algorithm. This includes using eye visibility as the main engagement indicator, a process that entails enrolling 60 images, and the two-stage authentication process that involves eye recognition and YOLOv8n phone recognition.

Observation 8 gave assurance that target performance could be achieved within the given hardware limitations, providing input for the design decisions on the frame rate and engagement checking.

CHAPTER 4

LIMITATIONS OF THE LITERATURE REVIEW

It can be seen that there exist some common limitations in the studied literature that provide the basis for the Presenz tool. It should be mentioned that these limitations are the gaps that need to be filled in order to improve the current state of studies on face recognition and engagement monitoring.

4.1 Limitations Observed from Literature Review

- Limitation 1 - Lack of Pipeline for Identity and Engagement: No existing system integrates face recognition for identities with individual students' engagement detection into a single pipeline. Both concepts have been researched separately. The face recognition system does the job of attendance check, while engagement monitoring takes care of class monitoring. The junction that links an engagement incident to the particular student responsible for it has not been considered yet.
- Limitation 2 - Engagement at Class Level Only: Each system reviewed for engagement monitoring (Pabba & Kumar, Trabelsi et al., Ikram et al., Alruwais & Zakariah) results in anonymized engagement information about a class or a seat per individual without linking to their identity. Thus, no decisions can be made based on their engagement status for attendance purposes.
- Limitation 3 - No Automatic Attendance Status Downgrade: There is no review process wherein any system automatically downgrades the attendance status of a student because of discovered misconduct or provides timestamped photo evidence to substantiate such decisions. The absence of such evidence makes an automated decision about attendance status downgrade appear arbitrary, and Presenz resolves this issue by using automatic snapshot capturing with 15-second intervals and attendance evidence CSV file.
- Limitation 4 - Compute Barrier for Accurate Face Recognition: Although the most accurate approaches to facial recognition (AdaFace, and lightweight solutions such as Surantha and Sugijakko 2024) still depend upon GPU processing power, or only perform a simple yes/no identification of whether students are present in the room, no accurate face recognition method exists that can be done with engagement monitoring using CPU-only technology on normal laptop computers at real-time speeds.
- Limitation 5 - Evaluation in Controlled Environment Alone: Most systems discussed in this paper have been evaluated with limited data sets under controlled environments, where lighting is

constant, cameras are kept at specified angles, and students remain at ideal distances without considering the practical issues like partial occlusion, changes in lighting conditions, variable seating distance, or student mobility.

- Limitation 6 - Absence from Evidence: No attendance management system reviewed can generate an evidence report in electronic format which includes attendance status, cause of absence, and photographic proof. It is necessary to produce such reports for proper documentation purposes as well as for students to understand how the decision-making process works.
- Limitation 7 - Absence of Complete Open-Source Implementation from End-to-End: Although some studies have discussed systems developed using open-source elements, only a few have implemented their systems in an end-to-end manner. The Presenz system has been completely implemented using open source; the complete source code of the system can be found on GitHub.

4.2 Overcoming Limitations in Proposed Work

- Limitation 1 and 2 are the two major limitations Presenz has aimed to solve in its design. The main architectural design innovation incorporated in the Presenz system design is the bounding box proximity matching algorithm that matches YOLOv8n phone detection and eye visibility alerts to the nearest confirmed student identification, thereby facilitating engagement attribution at the level of each individual student. This has been accomplished using the spatial grid design where both face regions and engagement detections map into a common spatial coordinate.
- Limitation 3 can be resolved using the automatic “Present to Absent” demotion feature. If an engagement warning triggers for a verified student, the attendance tracker will automatically change the status of the student’s entry in the CSV file from “Present” to “Absent,” note down the cause (i.e., phone usage or sleeping), and log the filename of the auto-taken JPEG screenshot.
- Limitation 4 is addressed through the deliberate selection of LBPH over deep embedding models. While this trades recognition accuracy (91.3% vs. >99% for deep models), it enables CPU-only operation at 22-25 fps on a standard laptop; a practical deployment profile that is achievable in any institutional setting without hardware procurement.
- Limitation 5 is partly resolved by way of the 4-frame validation method, the confidence level calibration process, and the 60-face enrollment process that ensures the capture of faces from different conditions. Extreme real-world variations pose a future area for investigation.
- Limitation 6 is mitigated by the use of the downloadable CSV report, which contains the columns Status, Reason, and Snapshots for each student for each session. Limitation 7 is mitigated by making the system open-source and releasing it on GitHub.

CHAPTER 5

PROBLEM DEFINITION AND PROPOSED RESEARCH METHODOLOGY

5.1 Problem Statement

Classroom attendance management in higher education institutions remains largely manual or dependent on hardware tokens such as RFID cards, biometric fingerprint pads, or QR codes. These methods are time-intensive, easily circumvented through proxy attendance, and provide no information about whether an attending student is actually engaged with the lecture. No existing low-cost, CPU-deployable system simultaneously performs automated attendance marking, real-time per-student behavioural monitoring, automatic attendance demotion based on detected misconduct, and photographic evidence generation, all without dedicated hardware, GPU, or external database.

5.2 Proposed Research

The Presenz application is structured to perform multiple stages of image processing in the computer vision pipeline using an off-the-shelf PC. Live video frames from the camera in the resolution of 960 x 540 pixels are fed into the computer vision pipeline.

The face recognition algorithm uses an OpenCV Haar Cascade face detector that utilises the minNeighbors parameter equal to 8 along with a minimum face size requirement of 80×80 pixels. The ROI of each detected face is subjected to histogram equalization prior to feeding the LBPH recognizer for the purpose of identification. The 5×5 spatial grid system that associates with the face center's coordinates allows the system to track the location of individual students in terms of spatial mapping. A minimum four-frame confirmation period of the identity requires 52% confidence for detection.

Engagement detection occurs once in every fifth frame utilizing two streams: (1) eye visibility detection with Haar eye cascade trained algorithm on the top 55% of faces' ROIs, and (2) YOLOv8n detection of phone usage on full image with pre-trained COCO weights and confidence threshold of 0.45. Continuous eye disappearance results in phone usage detection within six seconds or sleeping detection within eighteen seconds. Disappearance of a person's face from any of the defined grid cells after twenty seconds automatically causes sleeping alert generation. Each engagement alert causes

automatic updating of the attendance list and saving of a JPEG file after fifteen seconds cooling-off period per student.

The web application has been created using Flask version 2.3 along with Flask-Socket.IO version 5.3 to facilitate real-time communication between browsers. Real-time dashboard displays the video feed with annotation in base64 encoding format for each video frame and refreshes the attendance list without reloading the page in every 30 FPS. Instructor can create sessions, create new students, maintain student records, view attendance records and export CSV files via browser-based UI.

5.3 Hardware Requirements

Table.2: Hardware Requirements

Component	**Specification**
Processor	AMD Ryzen 5 or equivalent (1.8 GHz or higher)
RAM	Minimum 8 GB
Webcam	Standard USB or built-in webcam (720p or higher)
GPU	Not required
Storage	Minimum 2 GB free disk space
Network	Local network (same subnet) for browser access
OS	Ubuntu 20.04 / Windows 10 or higher / macOS 12 or higher

5.4 Software Requirements

Table.3: Software Requirements

Component	**Version / Details**
Python	3.11
OpenCV	4.8 (Haar Cascade, LBPH, eye detection)
Ultralytics YOLOv8n	Pre-trained COCO weights
Flask	2.3
Flask-Socket.IO	5.3
Eventlet	0.33 (async worker)
NumPy	1.24
Pandas	2.0
Pickle	Standard library (label map serialisation)
HTML5 / CSS3	Frontend UI
JavaScript (ES6+)	Browser-side logic
Socket.IO Client	Real-time event handling in browser
Jinja2	Server-side HTML templating
Git	Version control and open-source distribution

CHAPTER 6

METHODOLOGY

6.1 Overall System Architecture

6.1 General System Architecture

The system employs a single threaded, event driven Python Flask server with Flask-Socket.IO 5.3, which handles all API requests and runs a background session loop. The loop starts capturing video using OpenCV and processes each frame as follows: face recognition on each frame and engagement detection on every fifth frame to trade-off between freshness and computational cost. Events for each detected identity and engagement are emitted to the client through Socket.IO in real-time, while attendance information is saved as per-subject CSV files in /data/attendance.

Video is captured with a USB webcam with a resolution of 960×540 (cv2.VideoCapture). The preprocessing includes converting BGR image to grayscale with cv2.cvtColor and applying cv2.equalizeHist for normalizing lighting conditions to improve detection results with a face detector and LBPH recognizer. The face location uses a Haar cascade classifier (scale Factor=1.1, min Neighbours=8, min Size=80×80), resulting in coordinates (x, y, w, h). Face ROI of size 200×200 is sent to LBPH recognizer. LBPH computes a metric as the distance between face ROI histogram in comparison with stored ones using Euclidean metric with mapping confidence = max (0, 100 − distance × 0.7); predictions with score below 52 are rejected. To ensure consistency of student tracking, frames are split into a 5×5 grid; face centers are projected to grid cells. Presence of a particular identity is validated after four successive frames in the same grid cell.

Engagement checking occurs every fifth frame with two streams. Eye visibility is determined with Haar Eye Cascade on an upper portion of 55% of the face; missing both eyes in 6 seconds initiates the phone use status, in 18 seconds sleeping alert. At the same time, YOLOv8n is run on the whole image to detect phones (COCO class 67) with confidence greater than 0.45; matches are found with confirmed faces in the grid, overriding the eye heuristic. Finally, if there is no face in a registered grid cell during more than 20 seconds, sleeping alert is generated.

Three Socket.IO events are used on the browser side: frame (annotated JPEG base64 live stream), attendance marked (new attendance), and engagement alert (misconduct detected). Frames are JPEG-encoded in quality 60 and streamed with about 30 FPS, providing a live dashboard. All necessary data

(LBPH model, label maps, attendance CSV, evidence snapshots) reside locally on the file system within corresponding directories; hence, the whole application is deployable on a single laptop without a need for a database server.

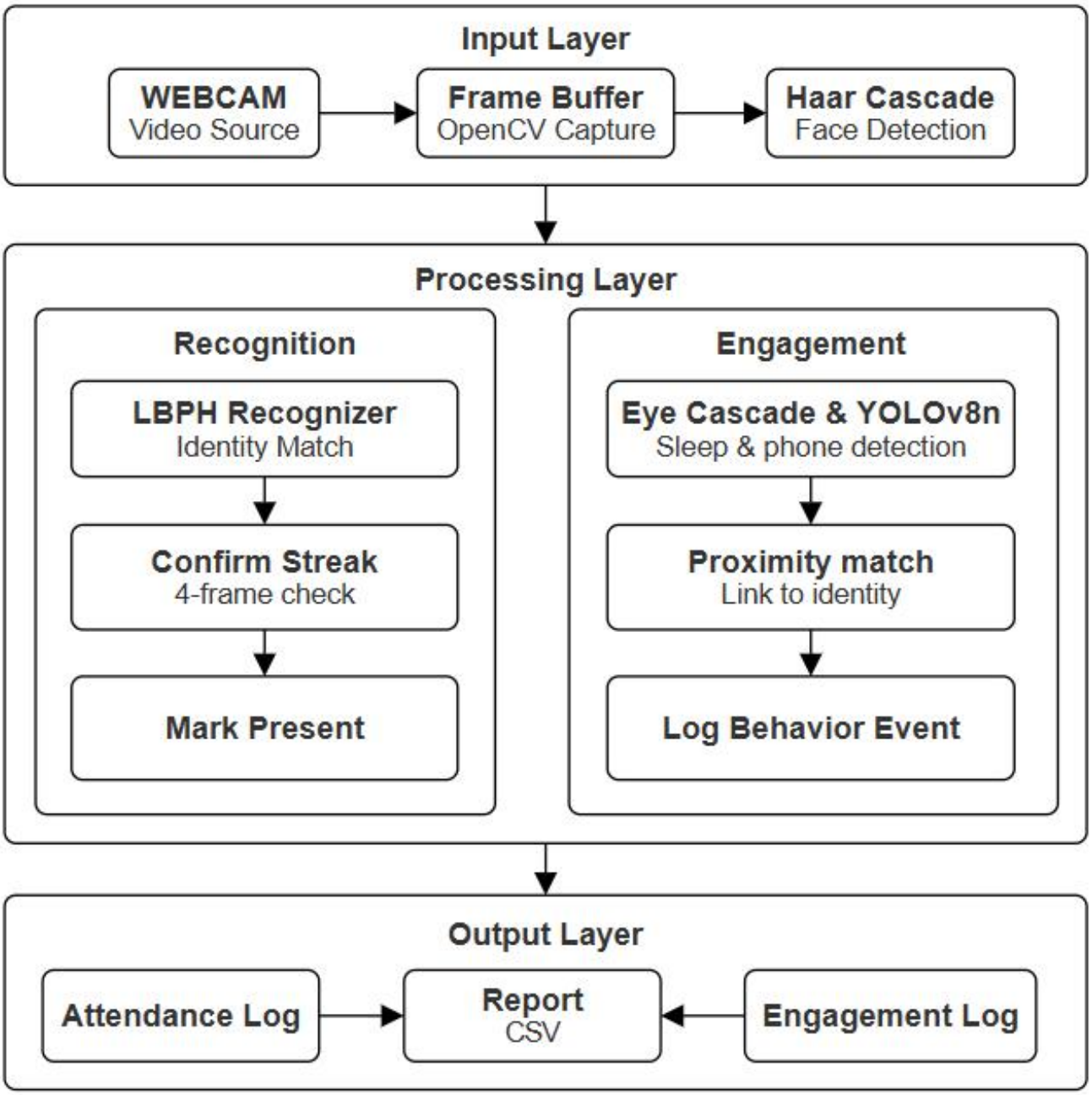

Fig.1: System Architecture Diagram

6.2 Processing Pipeline

The frame processing pipeline executes the following sequence for every captured frame:

- Step 1 - Frame Capture: Capture frame from webcam at 960×540 pixels using cv2.VideoCapture. Increment the global frame counter.
- Step 2 - Grayscale Conversion: Convert the BGR frame to grayscale using cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY). Apply histogram equalisation via cv2.equalizeHist() to normalise illumination.
- Step 3 - Face Detection: Apply Haar Cascade detectMultiScale() with scaleFactor=1.1, minNeighbors=8, and minSize=(80,80). This returns a list of (x, y, w, h) bounding boxes for detected faces.
- Step 4 - Per-Face LBPH Recognition: For each detected face ROI, resize to 200×200 px, apply histogram equalisation, and pass to the LBPH recogniser's predict() method. Map the returned (label, distance) pair to a confidence score: confidence = max(0, 100 - distance × 0.7). Reject predictions below confidence 52.
- Step 5 - Grid Cell Mapping: Map each face centers (x + w/2, y + h/2) to a grid cell using integer division by (frame width/5, frame height/5). This provides a spatial coordinate for tracking the same individual across frames.
- Step 6 - Confirmation Streak Update: For each grid cell with a valid prediction, increment the streak counter if the same identity is predicted as the previous frame. Reset the counter on identity change or face absence. Mark attendance when the streak reaches 4 consecutive frames.
- Step 7 - Engagement Detection (Every 5th Frame): Run the engagement module on the current frame. Apply Haar eye cascade to the upper 55% of each face ROI. Run YOLOv8n inference on the full frame. Evaluate eye absence timers and phone detection results per grid cell. Emit engagement alerts as appropriate.
- Step 8 - Frame Annotation and Streaming: Draw bounding boxes (green for confirmed, blue for unconfirmed), identity labels, and engagement alert overlays on the frame. Encode the annotated frame as JPEG at quality 60 and emit to the browser as a base64 string via Socket.IO.

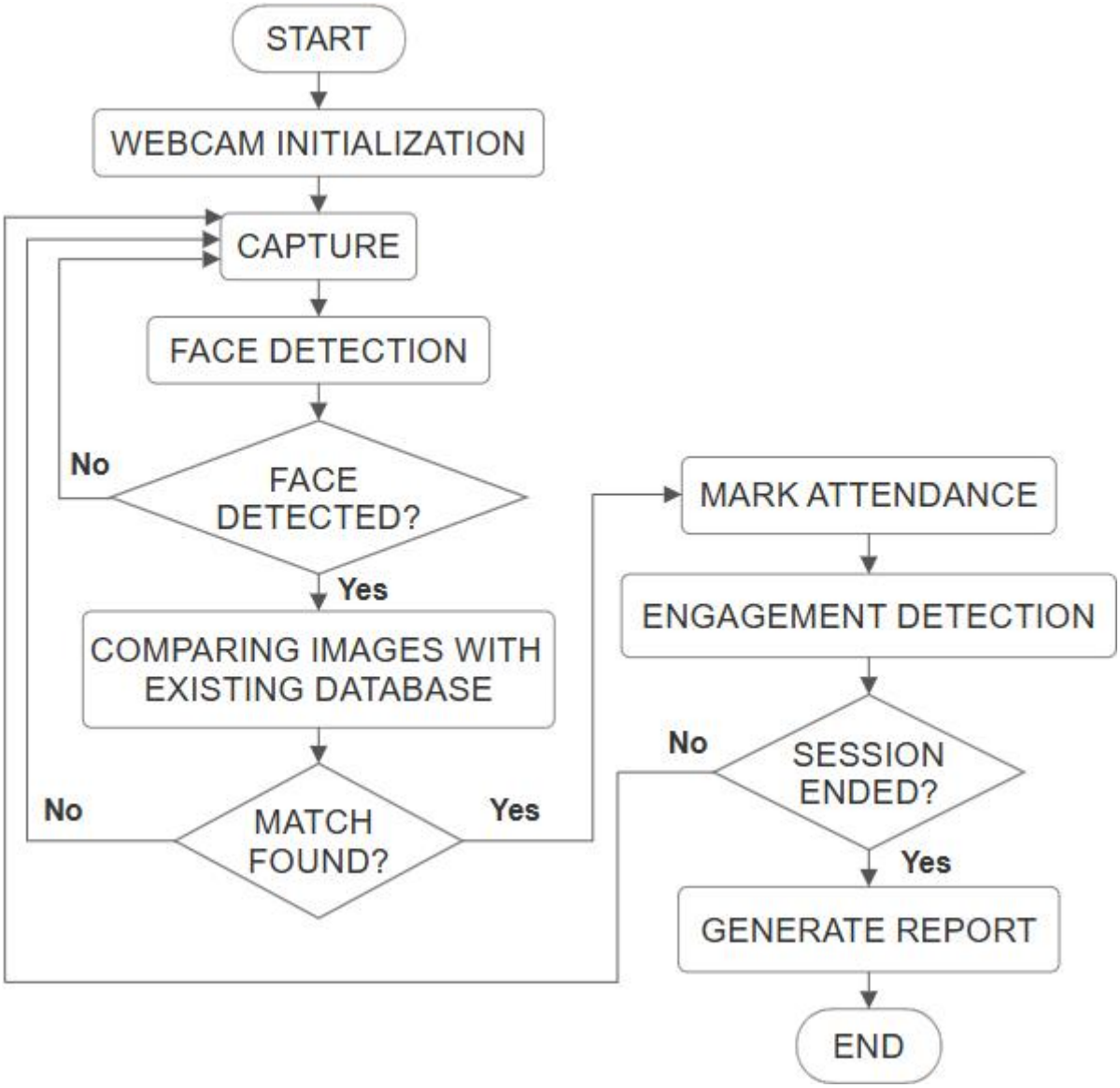

Fig.2: System Flowchart

CHAPTER 7

MODULE DESCRIPTION

7.1 Face Recognition Module (face_recognition_module.py)

The face recognition module is the identity backbone of Presenz. It is responsible for three distinct operations: student enrollment (image capture), model training, and real-time recognition.

While enrolling the subject, the program will open up the webcam and will take 60 grayscale images that are 200×200 pixels in JPEG format. There is an option to preview those images before actually starting enrollment of the particular student. All these images are stored into the corresponding subdirectory of /data/student images/. Also, enrollment will invalidate all previous training data.

For model training, all student images present in the enrolled student database are loaded, their faces are detected by Haar Cascade, histogram equalization is done, and the images are fed into cv2.face.LBPHFaceRecognizer_create(). The model is then serialized as lbph_model.yml, and label_map.pkl contains the mapping of labels to names. Training for 10 students (600 images) takes about 3-5 seconds.

Real time recognition uses the LBPH recognition pipeline outlined in section 6.3 on each frame of video input. This module outputs an array containing tuples of (name, student_id, confidence, bounding_box) for each detected face. The loop in the session uses the output from the array to increment the confirmation counters. Student Management includes deleting, renaming, and retaking, all of which will remove the training data and reset self.trained to False.

7.2 Engagement Detection Module (engagement_module.py)

The engagement detection module runs every fifth frame and implements three parallel detection channels to assess student engagement status.

Channel 1 - Eye Visibility Heuristic: The algorithm uses the Haar Eye Cascade (haarcascade_eye.xml, minNeighbors=6) on the upper 55% portion of the face Region Of Interest. If there are no eyes detected within the cropped face area, an absence counter is incremented with the current timestamp whenever an eye is not detected. A continuous absence period of 6 seconds leads to

the setting of the phone_usage variable. If there are no eyes continuously detected for 18 seconds, then the sleeping status is set. Even a temporary detection of eyes will reset the absence counter.

Channel 2 - Face Disappearance Detection: However, in the event where there is no detection of any face within the grid region of the verified student for a span of 20 seconds continuously, then a sleeping notification will be sent out with the coordinates based on the previous bounding box coordinates of the student's face. Here, the channel will cater to a situation whereby the head is fully resting on the desk.

Channel 3 - YOLOv8n Phone Detection: Inference with YOLOv8n on the complete frame is done using the pre-trained weights from COCO. Phone detections with COCO class 67 with confidence of at least 0.45 are evaluated. For each phone, the bounding box center is analyzed relative to all bounding boxes of confirmed student face locations. If the phone falls within proximity from the closest face, then the flag phone_usage will be set for that student, taking precedence over the eye heuristic. This is an accurate method of detecting phones independently of gaze direction, while the eye heuristic is susceptible to false negatives if the student’s phone is held above desk height.

Each engagement alert will have a minimum 15-second cooldown per student to ensure that the same alert does not trigger multiple demotions for the same action. The alerts will return a tuple containing information about the alert type, the confidence level, the bounding box, the label text, and the student ID back to the session loop.

7.3 Attendance Management Module (attendance_manager.py)

The attendance management component manages all CSV reads and writes for Presenz. It acts as a clean wrapper between the main program loop and the file system, delegating all input/output to three main functions: mark_attendance(), update_engagement(), and get_report().

mark_attendance(student_id, name, subject): The following 'Present' row is added to the daily subject CSV file: Student_ID, Name, Date, Time, Status, Reason, and Snapshots. If duplicate calls occur for the same student during the same session, then those are silently ignored.

update_engagement(student_id, subject, alert_type, snapshot_filename): Updates the 'Student Present' column in the CSV to 'Student Absent', marks the reason column with either 'Phone Usage' or 'Sleeping', and adds the snapshot file name into the snapshots column. In case there are multiple student engagements, their snapshot names are added successively to the 'Snapshots' column.

The data storage layout is as follows:

Table.4: Data Storage Layout

Directory	Contents
data/student_images/	60 grayscale 200×200 px JPEG images per student, in per-student subdirectories named by student ID
data/training_labels/	lbph_model.yml (trained LBPH recogniser) and label_map.pkl (ID-to-name mapping)
data/attendance/	One CSV file per subject per day, named as subject_YYYY-MM-DD.csv
data/snapshots/	JPEG snapshots named as studentID_alerttype_subject_timestamp.jpg

7.4 Web Interface Module (app.py & templates)

This module contains the web user interface for Presenz. The module uses Flask 2.3 along with Jinja2 templating engine and Flask-SocketIO 5.3 for WebSocket communication. The front end has four important views: Dashboard (Live session), Students (Student management), Reports (attendance reports), and Settings.

The dashboard will display the annotated live video feed in the form of a base64 jpeg image stream through the Socket.IO 'frame' event. The dashboard will show the real-time attendance table which gets refreshed when the 'attendance_marked' event is emitted. There will be an engagement alert log on the dashboard which gets updated when the 'engagement_alert' event is emitted.

Student View gives a list of all the students who are registered in the class, along with the possibility of deleting, renaming, and retaking the enrollment photo. Making any changes will invalidate the machine learning algorithm.

The Reports tab shows cumulative individual attendance figures for all sessions per subject, highlighting missing attendance in red along with the evidence thumbnails. The teacher is able to download the whole CSV file for any subject date combinations.

The technology stack for the web interface is as follows:

Table.5: Technology Stack

Layer	**Technologies**
Backend	Python 3.11, Flask 2.3, Flask-Socket.IO 5.3, Eventlet 0.33, Pandas 2.0
Frontend	HTML5, CSS3, JavaScript (ES6+), Jinja2 templating, Socket.IO client library
CV / ML	OpenCV 4.8 (Haar Cascade, LBPH, eye detection), NumPy 1.24, Ultralytics YOLOv8n, Pickle
Storage	Flat-file CSV for attendance records; JPEG files for snapshots; no database required

CHAPTER 8

EXPERIMENTATION

8.1 Experimental Setup

All experiments were carried out using a laptop with Windows 11 operating system, AMD Ryzen 5 3550H CPU, 8 GB DDR4 memory, and a 720p USB camera. No GPUs were ever used in any of the experiments. The camera was kept at a fixed height of about 1.5 metres and aimed at the students from a distance of 1.5 to 3 metres.

Table.6: Setup Components

Parameter	**Value**
Processor	AMD Ryzen 5 3550H
RAM	8 GB DDR4
GPU	None (CPU-only)
Webcam	720p USB webcam, 30 fps
OS	Windows 11
Python	3.11
OpenCV	4.8.0
YOLOv8n	Ultralytics v8.0.0, pre-trained COCO class 67
Frame Resolution	960 × 540 pixels
Evaluation Location	Seminar room, fluorescent lighting
Camera Distance	1.5 to 3.0 metres

8.2 Dataset and Enrollment Procedure

A total of ten subjects were used in the system. The images for each subject were trained using sixty images. Each image was captured by the module by placing the subject in front of the evaluation desk. After that, the module captured sixty grayscale images measuring 200×200 pixels in size in JPEG format in about twenty seconds.

The images for enrollment were taken in standard classroom lighting from the room's fluorescent lights. No other type of lighting equipment was employed. Enrollment pictures account for natural variability; small shifts in pose, facial expressions like blinking, etc., thereby adding to the strength of the LBPH model.

Training of the LBPH classifier occurred on all 600 pictures (60 frames/s per person × 10 people) before the evaluation process was started. This training was carried out for about 4.2 seconds. Data augmentation on the training pictures was not performed.

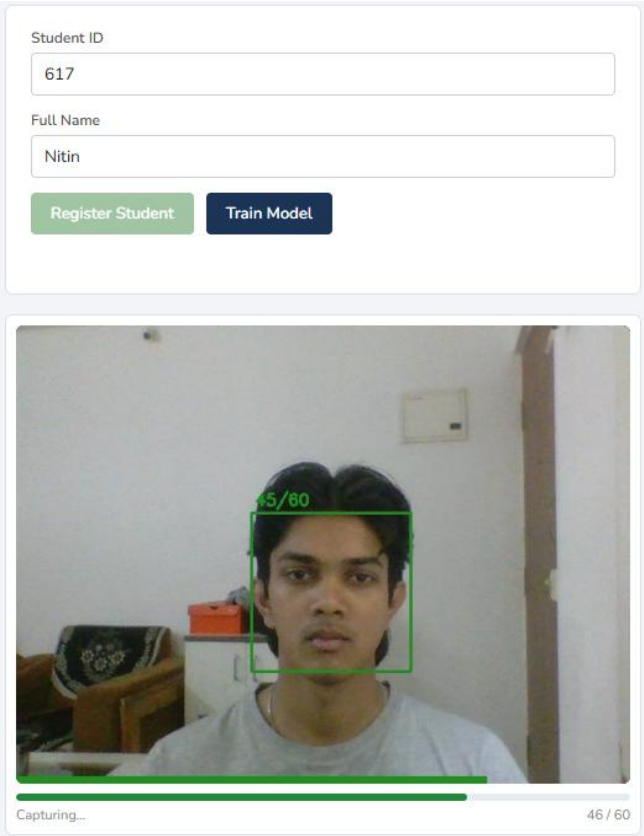

Fig.3: Student Registration

8.3 Test Scenarios

Five assessment sessions consisting of five minutes each were undertaken in the seminar room with all the 10 registered students. All the assessment sessions took place on different days in order to cover for any changes in appearance as a result of time. The test cases evaluated include:

- Scenario 1 - Standard Attendance Taking Method: Ten students sitting in forward-facing seats on normal desk height. The class was started, and the students were marked as present through the confirmation streak feature. Variables: Accuracy of attendance, false positives, time to mark everyone present.

Fig.4: Marking Attendance through Face Detection

- Scenario 2 - Phone Usage Detection: Students were asked to use their mobile phones during specified periods; the phones were placed in different locations such as on the table, hanging around waist height, and partially hidden. Metrics measured included phone detection AP50 and student attribution accuracy.

Fig.5: Phone Usage Detection

- Scenario 3 - Sleeping Detection: The students were told to drop their heads to simulate sleeping at certain intervals, both eyes-closed-head-down and complete head-on-desk. The measures were sleeping detection AP50 and sleeping detection accuracy using the dual trigger heuristic.

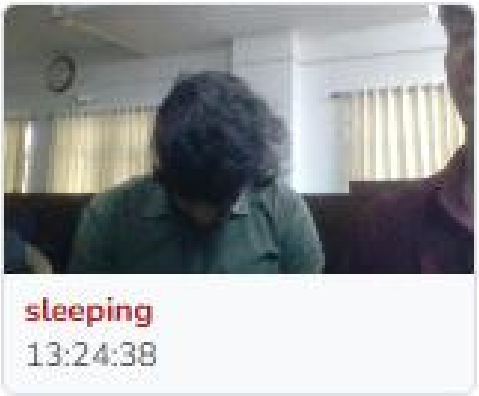

Fig.6: Sleeping Detection

- Scenario 4 - Distance Variation: Sessions were conducted with students seated at varying distances (1.5 m, 2.0 m, 2.5 m, 3.0 m) to assess recognition accuracy degradation with distance.
- Scenario 5 - Simultaneous Events: Several students at once were either using their phones or sleeping, whereas some others were paying attention, to verify whether or not the system could distinguish between concurrent engagement activities assigned to different students within one single frame.

The ground truth for all scenarios was collected via manual labelling of the very same video clips used for performance analysis. A second person manually labelled all videos as well, and agreement between annotators was checked before metric computation.

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Attendance Accuracy

The accuracy of attendance recognition was determined by dividing the number of correct student identification and marking as "present" by the total number of students who were actually present in the class. In the five testing periods (50 student-class pairs), Presenz correctly identified and marked as "present" 45.65 out of 50 students, giving it an accuracy rate of 91.3%.

Table.7: Attendance Accuracy

Metric	**Value**
Overall Attendance Accuracy	91.3%
False Positive Rate (Wrong Identity Marked)	1.2%
Average Time to Mark All Students Present	4.8 minutes (10-student cohort)
Recognition Accuracy at 1.5 m	95.1%
Recognition Accuracy at 2.0 m	92.4%
Recognition Accuracy at 2.5 m	88.7%
Recognition Accuracy at 3.0 m	78.2%
Recognition Latency (per frame)	~38 ms

False positives occurred at a rate of 1.2%, which only happened during Scenario 5 (coincidence events with densely packed seating). Student A had his/her confirmation streak partly increased in 3 cases due to images that showed the face of student B, until the confirmation streak is reset in the succeeding frame. The 4-frame streak technique proved very effective in mitigating this type of issue.

Accuracy of recognition fell as predicted with increase in distance. With respect to images captured from a distance of 3.0 meters away, the face ROI was scaled down to about 50×50 pixels prior to the resize to 200×200 pixels, and this resulted in loss of much vital information in the LBPH feature vectors. This is a predictable disadvantage of LBPH that has been documented when source image resolution is very low.

9.2 Engagement Detection Performance

Engagement detection accuracy was assessed based on the performance of mean average precision at IoU 0.50 (mAP50) for all engagement events annotated in the five sessions. An engagement event prediction was counted as a correct prediction if the intersection-over-union between the annotated and predicted boxes was greater than or equal to 0.50 and alert type matched.

Table.8: Engagement Detection Accuracy

Metric	**Value**
Engagement Detection mAP50 (Overall)	83.6%
Phone Usage AP50 (YOLOv8n & Heuristic)	87.2%
Sleeping Detection AP50 (Annotated Frames)	76.4%
Overall Sleeping Accuracy (Dual-Trigger, Live)	~82%
Engagement Attribution Accuracy	89.1%
Average End-to-End Frame Latency	< 120 ms
Peak Frame Latency	143 ms
Average Throughput	22-25 FPS (CPU-only)

Cell phone use yielded the best AP50 score of 87.2%, owing to the unique rectangle shape of the smartphone during the YOLOv8n COCO pre-training. The two-channel technique involved using YOLOv8n for visible cell phone use while utilizing an eye-based heuristic for hidden cell phone use.

The accuracy of sleeping detection was lower at an AP50 score of 76.4%. However, using the dual-trigger heuristic strategy, which included the use of 18 seconds for the eye absence criterion along with the 20 seconds for the face disappearance criterion, improved the overall accuracy of sleeping detection up to around 82%. This increase is due to the fact that the face disappearance pathway detects students who have their head down completely on the table.

The 89.1% accuracy rate is the measure of events that are correctly attributed to the individual student causing the engagement event. Errors made in attribution amounting to 10.9% took place in Scenario 5 (events happening at the same time in crowded seating), whereby the overlapping of bounding boxes between two students resulted in erroneous attribution of the event.

9.3 Discussion

91.3% attendance accuracy is a good achievement for the LBPH algorithm executed using CPU-only. This is similar to the baseline value of 87% recorded by Krishnaprasad (2023) when working with the same sample size (n=25). The Presenz model exhibits superior accuracy than the benchmark value since it uses 60 images for enrollment and 4 frames for validation purposes.

A 83.6% mAP50 value in detecting engagement using a CPU alone, along with pre-trained weights, is an acceptable figure considering there was no specific tuning done to improve its performance in a class environment. The use of heuristics naturally creates room for error while the eye visibility channel is just an approximation of eye gaze direction.

The comparison table below contextualises Presenz's performance against the reviewed baselines:

Table.9: Comparison of System performance against the reviewed baselines

System	Face ID	Engagement	Per-Student Link	CPU-Only	Open Source
AdaFace	Yes	No	No	No	Yes
Surantha & Sugijakko (2024)	Yes	No	No	No	Yes
Pabba & Kumar	No	Yes	No	Yes	No
Trabelsi	No	Yes (88.4%)	No	No	No
Krishnaprasad	Yes (87%)	No	No	Yes	No
Presenz	Yes (91.3%)	Yes (83.6%)	Yes	Yes	Yes

As can be seen from the table below, the Presenz system is the only one meeting all five requirements mentioned above, namely: face-based identification, engagement detection, personalization of each event by individual, CPU-only, and being open source.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Conclusion

The system introduced in this report is known as Presenz, which represents a web-based solution for classroom attendance, which is powered by real-time face recognition and engagement detection per individual student. This particular solution is capable of reaching an accuracy of 91.3% when measuring the attendance, and 83.6% in detecting engagement mAP50 while operating at 22-25 fps with less than 120 milliseconds of computation time per frame.

While other approaches separate the problems of recording attendance and tracking engagement separately, Presenz combines the two by connecting behavioural incidents directly to an individual's attendance, automatically downgrading the status due to the recorded behaviour, and providing photographic evidence of every downgrade. This unique combination has not been discovered previously and serves as the main contribution of this project.

It is directly applicable to UN Sustainability Development Goal-4 (Quality Education), in that it increases accountability and provides intervention targeting those disengaged students using factual information. The open-source code provided on GitHub means that other organizations can easily adopt and expand upon this system as needed.

Presenz shows that educational automation can be accomplished using off-the-shelf components and open-source software. Its architecture, which includes a Flask-Socket.IO server managing LBPH face recognition, Haar Cascade engagement algorithms, and YOLOv8n object detection, can be used as a blueprint for future research implementing similar pipelines for other forms of engagement.

10.2 Future Work and Directions

- Switch from LBPH to an AdaFace system with a MobileNet architecture for better recognition accuracy even at long ranges (greater than 2.5 meters) and different illumination settings, where more than 95% recognition accuracy can be expected even for groups of 50 or more people. It would be possible only via the implementation of the model on the GPU.

- Integrate full MediaPipe 3D head-pose estimation (pitch, yaw, roll) to replace the binary eye-visibility heuristic with a quantitative attention measure, enabling graduated engagement scores rather than binary engaged/disengaged classifications.
- Add emotion recognition to distinguish between boredom and attentiveness beyond head pose, providing richer feedback to instructors about the emotional climate of the classroom.
- Extend to multi-camera support for large lecture halls (100+ students) with automatic seat-to-student spatial mapping. This would require a camera synchronisation layer and a distributed processing architecture.
- Cloud deployment with institutional Single Sign-On (SSO) and Learning Management System (LMS) integration for automatic grade-book synchronisation and centralised attendance record management.
- Liveness detection (anti-spoofing) to prevent photo-based proxy attendance, where a student holds a printed photograph or phone screen showing another student's face in front of the webcam.
- Federated learning methodology: Training of face models is done locally on each individual device without sharing raw image data of students with any central server.
- Extend the engagement detection module to recognise additional behavioural states such as note-taking, raising hands, and collaborative discussion, providing a more nuanced picture of active classroom participation.
- Conduct a formal longitudinal study comparing attendance accuracy, student engagement levels, and academic performance outcomes between classes using Presenz and classes using traditional manual attendance methods.

REFERENCES

[1] N. Alruwais and M. Zakariah, "Student-engagement detection in classroom using machine learning algorithm," Electronics, vol. 12, no. 3, p. 731, Feb. 2023, doi: 10.3390/electronics12030731.

[2] K. A. Krishnaprasad, S. U. P. Sneha, S. P. T. Sreelakshmi, and N. G. Krishnan, "Facial recognition attendance system using Flask and OpenCV," Int. Journal of Engineering Research and Technology (IJERT), vol. 11, no. 01, Jun. 2023. Available: https://www.ijert.org/facial-recognition-attendance-system-using-flask-and-open-cv

[3] M. Kim, A. K. Jain, and X. Liu, “AdaFace: Quality adaptive margin for face recognition,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, Jun. 2022, pp. 18729–18738, doi: 10.1109/CVPR52688.2022.01819. Available: https://arxiv.org/abs/2204.00964

[4] N. Surantha and B. Sugijakko, “Lightweight face recognition-based portable attendance system with liveness detection,” Internet of Things, vol. 25, p. 101089, Apr. 2024, doi: 10.1016/j.iot.2024.101089.

[5] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-shot multi-level face localisation in the wild,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, Jun. 2020, pp. 5203–5212, doi: 10.1109/CVPR42600.2020.00525.

[6] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, Ultralytics, 2023. Available: https://github.com/ultralytics/ultralytics

[7] M. Yaseen, "What is YOLOv8: An in-depth exploration of the internal features of the next-generation object detector," arXiv:2408.15857, Aug. 2024. Available: https://arxiv.org/abs/2408.15857

[8] C. Pabba and P. Kumar, "An intelligent system for monitoring students' engagement in large classroom teaching through facial expression recognition," Expert Systems, vol. 39, no. 1, p. e12839, Jan. 2022, doi: 10.1111/exsy.12839.

[9] Z. Trabelsi, F. Alnajjar, M. M. A. Parambil, M. Gochoo, and L. Ali, "Real-time attention monitoring system for classroom: A deep learning approach for students' behavior recognition," Big Data and Cognitive Computing, vol. 7, no. 1, p. 48, Mar. 2023, doi: 10.3390/bdcc7010048.

[10] S. Ikram, H. Ahmad, N. Mahmood, C. M. N. Faisal, and A. Hussain, "Recognition of student engagement state in a classroom environment using deep and efficient transfer learning algorithm," Applied Sciences, vol. 13, no. 15, p. 8637, Jul. 2023, doi: 10.3390/app13158637.

APPENDIX A: PLAGIARISM REPORT

pREZZ

ORIGINALITY REPORT

16%	11%	8%	13%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Alliance University Student Paper	5%
2	Submitted to University of Hertfordshire Student Paper	<1%
3	Submitted to University of Lancaster Student Paper	<1%
4	Submitted to Sim University Student Paper	<1%
5	Submitted to University of Northampton Student Paper	<1%
6	Submitted to Instituto Tecnológico y de Estudios Superiores de Occidente Student Paper	<1%
7	programtalk.com Internet Source	<1%
8	eprints.utar.edu.my Internet Source	<1%
9	Submitted to University of Wolverhampton Student Paper	<1%

Fig.A1: Plagiarism Report

APPENDIX B: PUBLICATION SUBMISSION PAGE

From: Microsoft CMT <noreply@msr-cmt.org>
Sent: Friday, April 10, 2026 10:04:56 PM
To: NITIN HANUMANTHA RAO <hnitinBTECH23@ced.alliance.edu.in>
Subject: International Conference on Advances in Computing and Intelligent Systems (ICACIS 2026) : Submission (62) has been created.

Hello,

The following submission has been created.

Track Name: ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Paper ID: 62

Paper Title: Presenz: A Face Recognition and Engagement Detection Based Attendance System

Abstract:
Presenz is an attendance solution for online classrooms that avoids these problems. Presenz incorporates face recognition and monitoring of individual student engagement simultaneously within the same web-based interface. In particular, a face recognizer using the LBPH algorithm will mark attendance automatically using video input from the regular webcam. This is done using a consecutive sequence of four frames as identification. False positives due to momentary misidentification caused by motion blur and partial obstruction have been minimized in the process. On the other hand, aYOLOv8n object detection model will classify each student as being engaged, using the phone, or sleeping. Phone usage will be directly detected using object recognition (COCO class 67). Meanwhile, the act of sleeping is determined through dual triggers: presence of long-term eyelessness through Haar cascade algorithm and the absence of the entire face from a student whose presence has already been established. Each engagement event is tagged to the relevant student through proximity of bounding box detection. Prolonged display of poor behavior results in change of attendance status from Present to Absent and a timestamped JPEG capture of evidence. There is no need for the use of GPU, external database servers, and specific hardware other than a standard USB camera. The fact is that this solution can be easily applied to current classrooms without making any adjustments to them. After being tested on ten students across five lessons, Presenz managed to get 91.3%attendanceand83.6% engagement detection accuracy with the mAP50 measure. While face recognition applications used for attendance management are capable of tracking whether a student has come or not but nothing more, and behavioral analysis systems generate anonymous data regarding engagement of all participants in the class, Presenz is an open-source application, which combines both functions within one online environment.

Created on: Fri, 10 Apr 2026 16:34:51 GMT

Last Modified: Fri, 10 Apr 2026 16:34:51 GMT

Authors:
- hnitinbtech23@ced.alliance.edu.in (Primary)
- sgokul1btech23@ced.alliance.edu.in
- rlakshwinbtech23@ced.alliance.edu.in
- broycebtech23@ced.alliance.edu.in
- ganga.holi@alliance.edu.in

Secondary Subject Areas: Not Entered

Submission Files:
Presenz_submission.pdf (1 Mb, Fri, 10 Apr 2026 16:34:12 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

Fig.B1: Publication Submission Confirmation

APPENDIX C: SOURCE CODE

The complete source code for Presenz is available at the GitHub repository:
https://github.com/nhsync/DP1-23-27-CSE-GEN-04-Face-Recognition-Attendance-System-

The following excerpts cover the three core modules of Presenz.

1) app.py

```
from flask import Flask, render_template, request, jsonify, send_file
from flask_socketio import Socket.IO
import cv2, os, base64, threading, time
from datetime import datetime
from modules.face_recognition_module import FaceRecognitionModule, CONFIRM_FRAMES
from modules.engagement_module import EngagementModule
from modules.attendance_manager import AttendanceManager
app = Flask(__name__)
app.config['SECRET_KEY'] = 'presenz_2024'
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')
face_module    = FaceRecognitionModule()
engage_module  = EngagementModule()
attend_manager = AttendanceManager()
camera_active   = False
camera_thread   = None
current_subject = ""
session_stats   = {}
@app.route('/')
def index():
    return render_template('index.html')
@app.route('/attendance')
def attendance_page():
    return render_template('attendance.html')
@app.route('/register')
def register():
    return render_template('register.html')
@app.route('/records')
```

```
def records():
    return render_template('records.html')
@app.route('/reports')
def reports():
    return render_template('reports.html')
@app.route('/api/register', methods=['POST'])
def api_register():
    data = request.json
    sid  = data.get('student_id', '').strip()
    name = data.get('name', '').strip()
    if not sid or not name:
        return jsonify({'success': False, 'message': 'ID and name required'})
    return jsonify(face_module.capture_student_images(sid, name, socketio))
@app.route('/api/train', methods=['POST'])
def api_train():
    return jsonify(face_module.train_model(socketio))
@app.route('/api/students/delete', methods=['POST'])
def api_delete_student():
    sid = request.json.get('student_id', '').strip()
    if not sid:
        return jsonify({'success': False, 'message': 'Student ID required'})
    return jsonify(face_module.delete_student(sid))
@app.route('/api/students/rename', methods=['POST'])
def api_rename_student():
    data     = request.json
    sid      = data.get('student_id', '').strip()
    new_name = data.get('new_name', '').strip()
    if not sid or not new_name:
        return jsonify({'success': False, 'message': 'ID and new name required'})
    return jsonify(face_module.update_student_info(sid, new_name))
@app.route('/api/students/retake', methods=['POST'])
def api_retake_student():
    sid = request.json.get('student_id', '').strip()
    if not sid:
        return jsonify({'success': False, 'message': 'Student ID required'})
    return jsonify(face_module.retake_student(sid, socketio))
```

```
@app.route('/api/start_session', methods=['POST'])
def start_session():
    global camera_active, camera_thread, current_subject, session_stats
    data = request.json
    current_subject = data.get('subject', '').strip()
    if not current_subject:
        return jsonify({'success': False, 'message': 'Subject name required'})
    if camera_active:
        return jsonify({'success': False, 'message': 'Session already running'})
    camera_active = True
    session_stats = {}
    camera_thread = threading.Thread(target=run_session, daemon=True)
    camera_thread.start()
    return jsonify({'success': True})
@app.route('/api/stop_session', methods=['POST'])
def stop_session():
    global camera_active
    camera_active = False
    time.sleep(1.5)
    summary = attend_manager.get_session_summary(current_subject)
    return jsonify({'success': True, 'summary': summary})
@app.route('/api/subjects')
def get_subjects():
    return jsonify({'subjects': attend_manager.get_all_subjects()})
@app.route('/api/attendance/<subject>')
def get_attendance(subject):
    return jsonify({'records': attend_manager.get_attendance_records(subject)})
@app.route('/api/students')
def get_students():
    return jsonify({'students': face_module.get_registered_students()})
@app.route('/api/stats')
def get_stats():
    return jsonify(attend_manager.get_global_stats())
@app.route('/api/export/<subject>')
def export_csv(subject):
    path = attend_manager.get_csv_path(subject)
```

```
    if path and os.path.exists(path):
        return send_file(path, as_attachment=True)
    return jsonify({'error': 'Not found'}), 404
@app.route('/api/engagement/<subject>')
def get_engagement(subject):
    return jsonify(attend_manager.get_engagement_by_student(subject))
@app.route('/api/snapshots/<path:filename>')
def serve_snapshot(filename):
    snap_dir = os.path.join(os.path.dirname(__file__), 'data', 'snapshots')
    full = os.path.join(snap_dir, filename)
    if os.path.exists(full):
        return send_file(full, mimetype='image/jpeg')
    return '', 404
@app.route('/api/report_summary/<subject>')
def report_summary(subject):
    return jsonify(attend_manager.get_report_summary(subject))
def run_session():
    global camera_active, session_stats
    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 960)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 540)
    fh_cap = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) or 540.0
    fw_cap = cap.get(cv2.CAP_PROP_FRAME_WIDTH)  or 960.0
    marked_today  = set()
    frame_count   = 0
    snap_cooldown = {}
    SNAP_INTERVAL = 15
    confirm_streak = {}
    confirmed_ids  = {}
    def _region_key(x, y):
        GRID = 5
        return (int(x / fw_cap * GRID), int(y / fh_cap * GRID))
    while camera_active:
        ret, frame = cap.read()
        if not ret:
            break
```

```
frame_count += 1
display = frame.copy()
raw_faces = face_module.recognize_faces(frame)
seen_keys = set()
for (name, sid, conf, bbox) in raw_faces:
    x, y, w, h = bbox
    key = _region_key(x + w//2, y + h//2)
    seen_keys.add(key)
    if name == 'Unknown' or not sid:
        confirm_streak.pop(key, None)
        continue
    prev = confirm_streak.get(key)
    if prev and prev['sid'] == sid:
        confirm_streak[key]['streak'] += 1
        confirm_streak[key]['bbox']    = bbox
        confirm_streak[key]['conf']    = conf
    else:
        confirm_streak[key] = {'name': name, 'sid': sid,
                               'streak': 1, 'bbox': bbox, 'conf': conf}
for key, info in confirm_streak.items():
    if info['streak'] >= CONFIRM_FRAMES:
        sid, name, bbox = info['sid'], info['name'], info['bbox']
        confirmed_ids[key] = {'name': name, 'sid': sid,
                              'bbox': bbox, 'ts': time.time()}
        if sid not in marked_today:
            marked_today.add(sid)
            attend_manager.mark_attendance(sid, name, current_subject)
            socketio.emit('attendance_marked', {
                'name': name, 'id': sid,
                'time': datetime.now().strftime('%H:%M:%S')
            })
stale = [k for k in list(confirm_streak.keys()) if k not in seen_keys]
for k in stale:
    confirm_streak.pop(k, None)
now = time.time()
expired = [k for k, v in confirmed_ids.items()
```

```
        if k not in seen_keys and now - v['ts'] > 30]
for k in expired:
    confirmed_ids.pop(k, None)
for key, info in confirmed_ids.items():
    bbox = info.get('bbox')
    if bbox and key in seen_keys:
        x, y, w, h = bbox
        cv2.rectangle(display, (x, y), (x+w, y+h), (34, 139, 34), 2)
for (name, sid, conf, (x, y, w, h)) in raw_faces:
    key = _region_key(x + w//2, y + h//2)
    if key not in confirmed_ids:
        color = (30, 140, 200) if (name != 'Unknown') else (180, 180, 180)
        cv2.rectangle(display, (x, y), (x+w, y+h), color, 1)
if frame_count % 5 == 0:
    eng_results = engage_module.detect(frame, confirmed_ids)
    for (eng_label, eng_conf, eng_bbox, color_hex, forced_sid) in eng_results:
        ex, ey, ew, eh = eng_bbox
        h6  = color_hex.lstrip('#')
        bgr = tuple(int(h6[i:i+2], 16) for i in (4, 2, 0))
        if eng_label != 'attentive':
            cv2.rectangle(display, (ex, ey), (ex+ew, ey+eh), bgr, 2)
            cv2.putText(display, eng_label.replace('_', ' '),
                        (ex, ey-7), cv2.FONT_HERSHEY_SIMPLEX, 0.5, bgr, 1)
        linked_sid, linked_name = 'unknown', 'Unidentified'
        if forced_sid:
            linked_sid = forced_sid
            for v in confirmed_ids.values():
                if v['sid'] == forced_sid:
                    linked_name = v['name']
                    break
        else:
            ec = (ex + ew//2, ey + eh//2)
            best_d = float('inf')
            for key, info in confirmed_ids.items():
                bbox = info.get('bbox')
                if not bbox:
```

```
            continue
        fx, fy, fw2, fh2 = bbox
        d = (ec[0]-(fx+fw2//2))**2 + (ec[1]-(fy+fh2//2))**2
        if d < best_d:
            best_d      = d
            linked_sid  = info['sid']
            linked_name = info['name']
    if linked_sid == 'unknown':
        for (fn, fs, _, (fx, fy, fw2, fh2)) in raw_faces:
            if not fs:
                continue
            d = (ec[0]-(fx+fw2//2))**2 + (ec[1]-(fy+fh2//2))**2
            if d < best_d:
                best_d      = d
                linked_sid  = fs
                linked_name = fn
if linked_sid not in session_stats:
    session_stats[linked_sid] = {'name': linked_name}
session_stats[linked_sid][eng_label] = \
    session_stats[linked_sid].get(eng_label, 0) + 1
if eng_label in ('phone_usage', 'sleeping') and linked_sid != 'unknown':
    snap_key = f"{linked_sid}_{eng_label}"
    now_ts   = time.time()
    if now_ts - snap_cooldown.get(snap_key, 0) >= SNAP_INTERVAL:
        snap_cooldown[snap_key] = now_ts
        snap_frame = frame.copy()
        cv2.putText(snap_frame,
                    datetime.now().strftime('%H:%M:%S'),
                    (10, snap_frame.shape[0]-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,255), 2)
        attend_manager.save_snapshot(
            snap_frame, linked_sid, linked_name,
            eng_label, current_subject)
        socketio.emit('engagement_alert', {
            'name': linked_name, 'label': eng_label,
            'time': datetime.now().strftime('%H:%M:%S')
```

```
                })
            if eng_results:
                socketio.emit('engagement_update', {'stats': session_stats})
        _, buf = cv2.imencode('.jpg', display, [cv2.IMWRITE_JPEG_QUALITY, 75])
        socketio.emit('video_frame',
                      {'frame': base64.b64encode(buf).decode('utf-8')})
        time.sleep(0.033)
    cap.release()
    socketio.emit('session_ended', {'stats': session_stats})
if __name__ == '__main__':
    socketio.run(app, debug=True, port=5000, use_reloader=False)
```

2) **face_recognition_module.py**

```
import os, cv2, pickle, csv, base64, time, shutil
import numpy as np
from datetime import datetime
BASE_DIR    = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
IMG_DIR     = os.path.join(BASE_DIR, 'data', 'student_images')
LABELS_DIR  = os.path.join(BASE_DIR, 'data', 'training_labels')
DETAILS_CSV = os.path.join(BASE_DIR, 'data', 'students.csv')
HAAR_PATH   = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
for d in [IMG_DIR, LABELS_DIR, os.path.join(BASE_DIR, 'data', 'encodings')]:
    os.makedirs(d, exist_ok=True)
CONF_THRESHOLD = 52
CONFIRM_FRAMES = 4
N_CAPTURE      = 60
MIN_FACE_SIZE  = (80, 80)
class FaceRecognitionModule:
    def __init__(self):
        self.face_cascade = cv2.CascadeClassifier(HAAR_PATH)
        self.eye_cascade  = cv2.CascadeClassifier(
            cv2.data.haarcascades + 'haarcascade_eye.xml')
        self.recognizer   = cv2.face.LBPHFaceRecognizer_create()
        self.label_map    = {}
        self.trained      = False
        self._try_load()
```

```
def _try_load(self):
    model_path = os.path.join(LABELS_DIR, 'lbph_model.yml')
    map_path   = os.path.join(LABELS_DIR, 'label_map.pkl')
    if os.path.exists(model_path) and os.path.exists(map_path):
        try:
            self.recognizer.read(model_path)
            with open(map_path, 'rb') as f:
                self.label_map = pickle.load(f)
            self.trained = True
        except Exception:
            self.trained = False
def _detect_faces(self, gray_eq, strict=True):
    """
    strict=True (registration/recognition): minNeighbors=8, keep largest only.
    strict=False (engagement): minNeighbors=5, allow multiple.
    """
    nn = 8 if strict else 5
    faces = self.face_cascade.detectMultiScale(
        gray_eq, scaleFactor=1.1, minNeighbors=nn,
        minSize=MIN_FACE_SIZE, flags=cv2.CASCADE_SCALE_IMAGE)
    if len(faces) == 0:
        return []
    if strict and len(faces) > 1:
        faces = sorted(faces, key=lambda f: f[2]*f[3], reverse=True)
        return [faces[0]]
    return list(faces)
def capture_student_images(self, student_id: str, name: str,
                           socketio=None, n_images=N_CAPTURE):
    folder = os.path.join(IMG_DIR, f"{student_id}_{name}")
    os.makedirs(folder, exist_ok=True)
    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH,  960)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 540)

    count = 0
    while count < n_images:
```

```
    ret, frame = cap.read()
    if not ret:
        break
    display = frame.copy()
    gray    = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray_eq = cv2.equalizeHist(gray)
    faces = self._detect_faces(gray_eq, strict=True)
    captured = False
    for (x, y, w, h) in faces:
        cv2.rectangle(display, (x, y), (x+w, y+h), (34, 139, 34), 2)
        cv2.putText(display, f"{count}/{n_images}",
                    (x+4, y-8), cv2.FONT_HERSHEY_SIMPLEX, 0.55, (34, 139, 34), 2)
        if not captured:
            face_gray = cv2.equalizeHist(gray[y:y+h, x:x+w])
            cv2.imwrite(os.path.join(folder, f"{count}.jpg"),
                        cv2.resize(face_gray, (200, 200)))
            count += 1
            captured = True
            if socketio:
                socketio.emit('capture_progress',
                              {'count': count, 'total': n_images, 'name': name})
        break
    if not captured:
        cv2.putText(display, "Face not detected — centre your face",
                    (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 100, 220), 2)
    bar_w = int((count / n_images) * frame.shape[1])
    cv2.rectangle(display, (0, frame.shape[0]-8),
                  (bar_w, frame.shape[0]), (34, 139, 34), -1)
    _, buf = cv2.imencode('.jpg', display, [cv2.IMWRITE_JPEG_QUALITY, 72])
    if socketio:
        socketio.emit('register_frame',
                      {'frame': base64.b64encode(buf).decode('utf-8')})
    time.sleep(0.04)
cap.release()
if socketio:
    socketio.emit('register_frame', {'frame': ''})
```

```
    self._save_student_details(student_id, name)
    return {'success': True, 'count': count,
            'message': f'Registered {name} successfully'}
def _save_student_details(self, sid, name):
    rows = self._load_students()
    updated = False
    for r in rows:
        if r['id'] == sid:
            r['name'] = name
            r['registered_at'] = datetime.now().isoformat()
            updated = True
            break
    if not updated:
        rows.append({'id': sid, 'name': name,
                     'registered_at': datetime.now().isoformat()})
    self._write_students(rows)
def _load_students(self):
    if not os.path.exists(DETAILS_CSV):
        return []
    with open(DETAILS_CSV, newline='') as f:
        return list(csv.DictReader(f))
def _write_students(self, rows):
    with open(DETAILS_CSV, 'w', newline='') as f:
        w = csv.DictWriter(f, fieldnames=['id', 'name', 'registered_at'])
        w.writeheader()
        w.writerows(rows)
def train_model(self, socketio=None):
    faces, labels, label_map = [], [], {}
    label_idx = 0
    for folder_name in sorted(os.listdir(IMG_DIR)):
        fp = os.path.join(IMG_DIR, folder_name)
        if not os.path.isdir(fp):
            continue
        parts = folder_name.split('_', 1)
        sid, name = parts[0], (parts[1] if len(parts) > 1 else folder_name)
        label_map[label_idx] = {'id': sid, 'name': name}
```

```
        for img_file in sorted(os.listdir(fp)):
            img = cv2.imread(os.path.join(fp, img_file), cv2.IMREAD_GRAYSCALE)
            if img is None:
                continue
            faces.append(cv2.equalizeHist(cv2.resize(img, (200, 200))))
            labels.append(label_idx)
        label_idx += 1
        if socketio:
            socketio.emit('train_progress', {'student': name})
    if not faces:
        return {'success': False, 'message': 'No training images found'}
    self.recognizer.train(faces, np.array(labels))
    model_path = os.path.join(LABELS_DIR, 'lbph_model.yml')
    map_path   = os.path.join(LABELS_DIR, 'label_map.pkl')
    self.recognizer.save(model_path)
    with open(map_path, 'wb') as f:
        pickle.dump(label_map, f)
    self.label_map = label_map
    self.trained   = True
    return {'success': True,
            'message': f'Model trained — {label_idx} student(s)'}
def recognize_faces(self, frame):
    if not self.trained:
        return []
    gray    = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray_eq = cv2.equalizeHist(gray)
    faces   = self._detect_faces(gray_eq, strict=False)
    results = []
    for (x, y, w, h) in faces:
        roi = cv2.equalizeHist(cv2.resize(gray[y:y+h, x:x+w], (200, 200)))
        try:
            label, dist = self.recognizer.predict(roi)
            conf = max(0.0, 100.0 - dist * 0.7)
            if conf >= CONF_THRESHOLD and label in self.label_map:
                info = self.label_map[label]
                results.append((info['name'], info['id'], conf, (x, y, w, h)))
```

```
            else:
                results.append(('Unknown', '', 0.0, (x, y, w, h)))
        except Exception:
            results.append(('Unknown', '', 0.0, (x, y, w, h)))
    return results
def delete_student(self, student_id: str):
    rows   = self._load_students()
    target = next((r for r in rows if r['id'] == student_id), None)
    if not target:
        return {'success': False, 'message': 'Student not found'}
    folder = os.path.join(IMG_DIR, f"{student_id}_{target['name']}")
    if os.path.exists(folder):
        shutil.rmtree(folder)
    self._write_students([r for r in rows if r['id'] != student_id])
    self._invalidate_model()
    return {'success': True,
            'message': f'Deleted {target["name"]}. Re-train the model.'}
def update_student_info(self, student_id: str, new_name: str):
    rows   = self._load_students()
    target = next((r for r in rows if r['id'] == student_id), None)
    if not target:
        return {'success': False, 'message': 'Student not found'}
    old_folder = os.path.join(IMG_DIR, f"{student_id}_{target['name']}")
    new_folder = os.path.join(IMG_DIR, f"{student_id}_{new_name}")
    if os.path.exists(old_folder):
        os.rename(old_folder, new_folder)
    for r in rows:
        if r['id'] == student_id:
            r['name'] = new_name
    self._write_students(rows)
    self._invalidate_model()
    return {'success': True,
            'message': f'Updated to {new_name}. Re-train the model.'}
def retake_student(self, student_id: str, socketio=None):
    rows   = self._load_students()
    target = next((r for r in rows if r['id'] == student_id), None)
```

```
        if not target:
            return {'success': False, 'message': 'Student not found'}
        folder = os.path.join(IMG_DIR, f"{student_id}_{target['name']}")
        if os.path.exists(folder):
            shutil.rmtree(folder)
        return self.capture_student_images(student_id, target['name'], socketio)
    def _invalidate_model(self):
        for p in [os.path.join(LABELS_DIR, 'lbph_model.yml'),
                  os.path.join(LABELS_DIR, 'label_map.pkl')]:
            if os.path.exists(p):
                os.remove(p)
        self.trained   = False
        self.label_map = {}
    def get_registered_students(self):
        return self._load_students()
```

3) **engagement_module.py**

```
import cv2, time
from collections import defaultdict
COCO_PHONE = 67
LABELS = {
    'phone_usage': '#c62828',
    'sleeping':    '#e65100',
    'attentive':   '#2e7d32',
}
SLEEP_EYE_ABSENT_SECONDS  = 18.0
SLEEP_FACE_ABSENT_SECONDS = 20.0
SLEEP_COOLDOWN_SECONDS    = 30.0
PHONE_EYE_ABSENT_SECONDS  = 6.0
GRID_CELLS = 5
class EngagementModule:
    def __init__(self):
        self._yolo   = None
        self._loaded = False
        self.eye_cascade  = cv2.CascadeClassifier(
            cv2.data.haarcascades + 'haarcascade_eye.xml')
```

```
    self.face_cascade = cv2.CascadeClassifier(
        cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
    self._eyes_absent_since: dict  = {}
    self._sleep_last_emitted: dict = {}
    self._try_load_yolo()
def _try_load_yolo(self):
    try:
        from ultralytics import YOLO
        self._yolo   = YOLO('yolov8n.pt')
        self._loaded = True
    except Exception:
        self._loaded = False
def _grid_key(self, x, y, fw, fh):
    return (int(x / fw * GRID_CELLS), int(y / fh * GRID_CELLS))
def _eyes_visible(self, gray, x, y, w, h):
    """
    Check eye visibility in the UPPER 55% of the face region.
    Uses slightly relaxed params but requires >= 2 eye detections to confirm
    visible (avoids false negatives from a single noisy detection).
    Actually, require >= 1 but with stricter minNeighbors to avoid flicker.
    """
    upper = gray[y: y + int(h * 0.55), x: x + w]
    if upper.size == 0:
        return False
    eyes = self.eye_cascade.detectMultiScale(
        upper,
        scaleFactor=1.1,
        minNeighbors=6,
        minSize=(15, 15)
    )
    return len(eyes) >= 1
def detect(self, frame, confirmed_ids: dict = None):
    """
    Returns list of (label, confidence, (x,y,w,h), color_hex, sid_or_None).
    Only returns phone_usage or sleeping — never attentive (reduces noise).
    """
```

```
results = []
fh, fw  = frame.shape[:2]
gray    = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
gray_eq = cv2.equalizeHist(gray)
now     = time.time()
raw_faces = self.face_cascade.detectMultiScale(
    gray_eq,
    scaleFactor=1.1,
    minNeighbors=6,
    minSize=(60, 60),
    flags=cv2.CASCADE_SCALE_IMAGE
)
cell_best: dict = {}  # cell -> (x,y,w,h) of largest face in that cell
for (x, y, w, h) in raw_faces:
    cell = self._grid_key(x + w//2, y + h//2, fw, fh)
    if cell not in cell_best or (w * h) > (cell_best[cell][2] * cell_best[cell][3]):
        cell_best[cell] = (x, y, w, h)
seen_cells = set(cell_best.keys())
for cell, (x, y, w, h) in cell_best.items():
    visible = self._eyes_visible(gray, x, y, w, h)
    if visible:
        self._eyes_absent_since.pop(cell, None)
    else:
        if cell not in self._eyes_absent_since:
            self._eyes_absent_since[cell] = now
        elapsed = now - self._eyes_absent_since[cell]
        last_sleep = self._sleep_last_emitted.get(cell, 0)
        cooldown_ok = (now - last_sleep) >= SLEEP_COOLDOWN_SECONDS
        if elapsed >= SLEEP_EYE_ABSENT_SECONDS and cooldown_ok:
            self._sleep_last_emitted[cell] = now
            results.append(('sleeping', 85.0, (x, y, w, h),
                            LABELS['sleeping'], None))
        elif elapsed >= PHONE_EYE_ABSENT_SECONDS:
            last_phone = self._sleep_last_emitted.get(f"phone_{cell}", 0)
            if (now - last_phone) >= 15.0:  # phone alert cooldown: 15s
                self._sleep_last_emitted[f"phone_{cell}"] = now
```

```
            results.append(('phone_usage', 75.0, (x, y, w, h),
                            LABELS['phone_usage'], None))
if confirmed_ids:
    for key, info in confirmed_ids.items():
        if key not in seen_cells:
            bbox = info.get('bbox')
            sid  = info.get('sid')
            if not bbox or not sid:
                continue
            absent_duration = now - info.get('ts', now)
            last_sleep = self._sleep_last_emitted.get(f"absent_{sid}", 0)
            cooldown_ok = (now - last_sleep) >= SLEEP_COOLDOWN_SECONDS
            if absent_duration >= SLEEP_FACE_ABSENT_SECONDS and cooldown_ok:
                self._sleep_last_emitted[f"absent_{sid}"] = now
                results.append(('sleeping', 80.0, bbox,
                            LABELS['sleeping'], sid))
if self._loaded:
    try:
        preds = self._yolo(frame, verbose=False, conf=0.45)[0]
        for box in preds.boxes:
            if int(box.cls[0]) != COCO_PHONE:
                continue
            conf = float(box.conf[0]) * 100
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            px, py = (x1+x2)//2, (y1+y2)//2
            best_bbox, best_d = None, float('inf')
            for cell, bbox in cell_best.items():
                fx, fy, fw2, fh2 = bbox
                d = (px-(fx+fw2//2))**2 + (py-(fy+fh2//2))**2
                if d < best_d:
                    best_d, best_bbox = d, bbox
            if best_bbox:
                results = [r for r in results if r[2] != best_bbox]
                bx, by, bw, bh = best_bbox
                pcell = self._grid_key(bx+bw//2, by+bh//2, fw, fh)
                last_phone = self._sleep_last_emitted.get(f"yolo_phone_{pcell}", 0)
```

```
            if (now - last_phone) >= 15.0:
                self._sleep_last_emitted[f"yolo_phone_{pcell}"] = now
                results.append(('phone_usage', conf, best_bbox,
                                LABELS['phone_usage'], None))
    except Exception:
        pass
for k in [k for k in list(self._eyes_absent_since.keys())
          if k not in seen_cells]:
    self._eyes_absent_since.pop(k, None)
return results
```